

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 2

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Tharish-K (191422014) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Tharish

AS
R

Backtracking search - Employee Task Assignment

Given:

Employees: E_1, E_2, E_3, E_4

Tasks:

1. Admission (A)
2. Accounts (AC)
3. Records (R)
4. IT support (IT)

constraints

* C1: $E_1 \rightarrow$ Admissions on Accounts only.

* C2: E_2 must be assigned before E_3 according to task order:

Admission $<$ Accounts $<$ Records $<$ IT support

* C3: only one employee can be assigned to each task.

* C4: $E_4 \neq$ Records.

* C5: Every employee gets exactly one task

Step 1: start with E_1

E_1 can only take Admissions ~~on Accounts~~.

Try: $E_1 \rightarrow$ Admissions

Constraint Check:

* E_1 is allowed to handle Admission $\rightarrow \checkmark$

* No task is duplicated

So, $E_1 =$ Admissions

Step 2: Assign E2

Remaining tasks:

Accounts, Records, IT support

Try:

E1 $\rightarrow$ Accounts

Check:

* Accounts is available $\rightarrow \checkmark$

* E1 is before E3 in task order $\rightarrow$ E1's task position = 1 $\rightarrow \checkmark$

So,

E1 = Accounts

Step 3: Assign E3

Remaining:

Records, IT support

Because E1 must be before E3:

* E1 = Accounts (position 1)

* E3 must be After Accounts

Therefore:

E3 $\rightarrow$ Records

Check:

* Accounts $<$ Records $\rightarrow \checkmark$

* Records is available $\rightarrow \checkmark$

So,

E3 = Records

Step 4: Assign E4

only remaining Task: IT support

check: * E4 $\neq$ Records $\rightarrow$ E4 gets IT support $\rightarrow \checkmark$

* IT support is available $\rightarrow \checkmark$

Therefore,

E4 = IT support

Backtracking Search Tree:

Start

Assign E1

E1 = Admission $\checkmark$

Assign E2

E2 = Accounts $\checkmark$

Assign E3

E3 = Records

Assign E4

E4 = IT support

$\rightarrow$ SOLUTION $\checkmark$

Constraint checking Table:

Step	Assignment	constraint check	Result
1	E1 $\rightarrow$ Admission	E1 can handle Admission	✓
2	E2 $\rightarrow$ Accounts	Accounts available	✓
3	E2 before E3	Accounts $\angle$ Records	✓
4	E3 $\rightarrow$ Records	Records available	✓
5	E4 $\rightarrow$ IT support	E4 cannot handle Records	✓
6	All employees assigned	one employee / task	✓

Final Valid Schedule:

Employee	Task	Position
E1	Admissions	1
E2	Accounts	2
E3	Records	3
E4	IT Support	4

Uniform Cost Search (UCS) for Security Patrol Robot

Given:

* Grid size = 6×6

* Start = S

* Goal = G

* Movement = Up, Down, Left, Right

* Normal movement cost = 1

* Mined corridor cost = 2

* Restricted cells cannot be entered

* Objective = Minimize total path cost

Assumed 6×6 grid

For demonstration, assume:

	C1	C2	C3	C4	C5	C6
R1	S	.	.	#	.	.
R2	#	.	.	#	.	.
R3	.	.	#	.	.	.
R4	.	#	#	.	#	.
R5	.	.	.	M	M	.
R6	.	.	.	.	.	G

Where :

* S = start

* G = goal

* . = Normal cell, movement cost = 1

* M = Monitored corridor, movement cost = 2

* # = Restricted cell

VCS Principle:

Uniform cost search always expands the node having the lowest cumulative path cost $g(n)$

Priority:

lowest $g(n) \rightarrow$ Highest $g(n)$

The evaluation function is: $f(n) = g(n)$

Where $g(n)$ is total cost from S to the current node.

Step by-step Node Expansion

Step 1: start.

Initial node:

$S = (1, 1)$

Priority Queue:

$[(S, 0)]$

Expand S

Possible moves: * Right $\rightarrow (1, 2)$, cost = 1 ✓

* Down $\rightarrow (2, 1)$, restricted X

step 2: Expand (1,2)

From (1,2):

* Right $\rightarrow (1,3) = 2$

* Down $\rightarrow (2,2) = 2$

* Left $\rightarrow$ S already visited

Queue: [(1,3), 2], [(2,2), 2]

step 3: Expand (1,3)

Possible moves: * Down $\rightarrow (2,3) = 3$

* Right $\rightarrow (1,4)$ restricted x

Queue = [(2,2), 2], [(2,3), 3]

step 4: Expand (2,2)

Possible moves: * Down $\rightarrow (3,2) = 3$

* Left $\rightarrow (2,1)$ restricted

* Right $\rightarrow (2,3) = 3$

Queue: [(2,3), 3], [(3,2), 3]

step 5: Expand (2,3)

Possible moves: * Down $\rightarrow (3,3)$ restricted x

* Right $\rightarrow (2,4)$ restricted x

Queue: [(3,2), 3]

step 6: Expand (3,2)

Possible moves: * Left $\rightarrow (3,1) = 4$

* Down $\rightarrow (4,2)$ restricted x

Queue: [(3,1), 4]

1870-1871

Jan 1st - 1st day of the year

Feb 1st - 2nd day of the year

Mar 1st - 3rd day of the year

Apr 1st - 4th day of the year

May 1st - 5th day of the year

Jun 1st - 6th day of the year

Jul 1st - 7th day of the year

Aug 1st - 8th day of the year

Sep 1st - 9th day of the year

Oct 1st - 10th day of the year

Nov 1st - 11th day of the year

Dec 1st - 12th day of the year

1872-1873

Jan 1st - 1st day of the year

Feb 1st - 2nd day of the year

Mar 1st - 3rd day of the year

Apr 1st - 4th day of the year

May 1st - 5th day of the year

Jun 1st - 6th day of the year

Jul 1st - 7th day of the year

step 13: Expand (6,2)

Right: $g(6,3) = 8 + 1 = 9$

Already in queue with cost 9, so no update

Queue: $[(6,3), 9], (5,4), 10]$

step 14: Expand (6,3)

Right $\rightarrow (6,4)$: $g(6,4) = 9 + 1 = 10$

Queue: $[(5,4), 10], (6,4), 10]$

step 15: Expand (5,4)

This is a monitored corridor.

Moving to (5,5) costs 2: $g(5,5) = 10 + 2 = 12$

Queue: $[(6,4), 10], (5,5), 12]$

step 16: Expand (6,4)

Move right: $g(6,5) = 10 + 1 = 11$

Queue: $[(6,5), 11], (5,5), 12]$

step 17: Expand (6,5)

Move right to goal: $g(6) = 11 + 1 = 12$

Queue: $[(6), 12], (5,5), 12]$

UCS - Pick from the queue first / steps.

Step 18 :

Step 18: $G(12) \rightarrow$ Goal reached

Optimal length:

$S \rightarrow (1,2) \rightarrow (1,3) \rightarrow (2,3) \rightarrow (2,2) \rightarrow (3,2) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (6,3) \rightarrow (6,4) \rightarrow (6,5)$

Final Answer:

Optimal path cost = 14

Therefore, uniform cost search finds the minimum-cost path from S to G with a total cost of 14.